Appliance Energy Prediction Using Deep Learning

Subject:

Multivariate Time-Series Forecasting

Objective:

Predict appliance energy consumption using environmental and temporal features using Machine Learning and Deep Learning models.

Name:

Jithara Siriwardana

Submission Date:

[11/08/2026]

Contents

List of Figures

List of Tables

1 Introduction

1.1 Background

Energy consumption forecasting is an important task in smart building management and intelligent energy systems. Accurate prediction of appliance-level energy usage helps improve energy efficiency, reduce electricity costs, and support sustainable energy consumption practices.

In this project, we focus on predicting appliance energy consumption using a real-world dataset that contains environmental, weather, and indoor sensor measurements collected at 10-minute intervals. The dataset includes variables such as temperature, humidity, wind speed, pressure, and indoor environmental conditions, which influence energy usage patterns.

To solve this problem, a complete machine learning pipeline is implemented, starting from data preprocessing and exploratory data analysis to feature engineering, model training, and evaluation. The core predictive model used in this project is a Long Short-Term Memory (LSTM) neural network, which is well-suited for time-series forecasting due to its ability to learn sequential dependencies in data.

1.2 Objectives

The main objectives of this project are:

- Perform exploratory data analysis (EDA) on the dataset to understand patterns and relationships between variables.
- Clean and preprocess the data by handling missing values, outliers, and duplicates.
- Engineer meaningful time-series features such as lag features, rolling statistics, and interaction features.
- Transform the dataset into sequences suitable for LSTM-based deep learning models.
- Develop a deep learning LSTM model for appliance energy consumption prediction.
- Optimize model performance using hyperparameter tuning techniques.
- Evaluate model performance using standard regression metrics such as MAE, RMSE, and R^2 score.
- Visualize model performance using plots such as actual vs predicted values and error distribution graphs.

2 Dataset Description and Overview

The dataset used in this project is the Appliance Energy Prediction dataset, which contains detailed measurements of environmental and household conditions recorded at 10-minute intervals. The dataset is designed to support time-series forecasting tasks, specifically predicting appliance-level energy consumption based on surrounding environmental factors.

The dataset includes a total of 19,735 observations and 28 features after preprocessing. The target variable is `Appliances`, which represents the energy consumption of household appliances in watts.

2.1 Input Features

The dataset consists of several categories of features.

Indoor Environmental Variables:

- Temperature and humidity measurements from multiple rooms, such as `T1`–`T9` and `RH_1`–`RH_9`, which capture internal climate conditions.

Outdoor Weather Conditions:

- Variables such as outside temperature (`T_out`), outside humidity (`RH_out`), wind speed, visibility, and dew point, which influence indoor energy usage.

Pressure and Random Variables:

- Atmospheric pressure (`Press_mm_hg`) and random variables (`rv1`, `rv2`) included for modeling robustness.

Lighting Information:

- Indoor lighting usage (`lights`), which may correlate with appliance usage patterns.

2.2 Target Variable

Appliances:

- `Appliances` is the dependent variable representing total energy consumption of household appliances.
- It is a continuous numerical variable and serves as the prediction target for the LSTM model.

2.3 Data Characteristics

- The dataset is time-series in nature, with observations collected at regular 10-minute intervals.
- All features are numerical after preprocessing and scaling.
- Missing values are handled using interpolation and forward/backward filling.
- Outliers are treated using the Interquartile Range (IQR) method.
- Feature scaling is applied using Min-Max normalization to improve neural network performance.

2.4 Suitability for Deep Learning

This dataset is well-suited for deep learning models such as LSTM because:

- It contains sequential time-dependent patterns.
- Energy consumption depends on past values and environmental trends.
- Relationships between variables are nonlinear and complex.
- The dataset is large enough to train deep learning models effectively.

3 Exploratory Data Analysis (EDA)

3.1 Overview

Exploratory Data Analysis (EDA) is a crucial step in the machine learning pipeline that helps to understand the structure, patterns, and relationships within the dataset before model development. In this project, EDA was performed on the appliance energy consumption dataset to gain insights into feature distributions, correlations, missing values, and overall data behavior.

The main goal of EDA is to identify meaningful patterns that can improve feature engineering and model performance, particularly for time-series forecasting using LSTM networks.

3.2 Dataset Inspection

The dataset contains 19,735 records and 28 numerical features after preprocessing. Initial inspection was performed using `.info()` and `.describe()` functions to understand:

- Data types of each feature.
- Presence of missing values.
- Statistical distribution of variables.
- Range of values across features.

Key Observations:

- All features are numeric after preprocessing.
- No missing values remain after interpolation and cleaning.
- The target variable `Appliances` shows significant variation, indicating dynamic energy usage patterns.

3.3 Missing Value Analysis

Missing value analysis was conducted to ensure data quality before model training. Missing values were identified using `df.isnull().sum()`.

Missing Data Handling Methods:

- Linear interpolation was applied for continuous variables.
- Forward fill and backward fill were used for remaining gaps.

After cleaning, no missing values remain in the dataset, ensuring data consistency for modeling.

3.4 Univariate Analysis

Univariate analysis was performed to understand the distribution of individual variables.

Key Findings:

- Most environmental features, such as temperature variables (`T1`–`T9`) and humidity variables (`RH_1`–`RH_9`), follow a normalized distribution after scaling.
- The target variable `Appliances` shows a right-skewed distribution, indicating that low energy consumption occurs more frequently than high consumption.
- Outliers were detected in several features, including wind speed, visibility, and outdoor temperature (`T_out`).
- These outliers were handled using the Interquartile Range (IQR) method.

3.5 Outlier Analysis

Outlier detection was performed using the IQR method:

$$\text{Lower bound} = Q1 - 1.5 \times IQR$$

$$\text{Upper bound} = Q3 + 1.5 \times IQR$$

Observations:

- Several environmental variables contained extreme values.
- Outliers were capped instead of removed to preserve time-series continuity.
- After treatment, feature distributions became more stable and suitable for deep learning models.

3.6 Correlation Analysis

A correlation matrix was generated to identify relationships between features.

Key Insights:

- Strong correlation exists between temperature-related features (`T1`–`T9`).
- Humidity features (`RH_1`–`RH_9`) also show moderate correlation patterns.

- Outdoor temperature (`T_out`) has a noticeable relationship with appliance energy usage.
- Random variables (`rv1`, `rv2`) show no meaningful correlation with the target variable.

These insights helped in understanding feature importance and redundancy.

3.7 Feature Distribution Analysis

Feature distribution plots were used to visualize the spread of key variables.

Observations:

- Temperature features are relatively stable across time.
- Humidity levels fluctuate more compared to temperature.
- Energy consumption shows periodic spikes, indicating usage patterns dependent on time and environmental conditions.

3.8 Key Insights from EDA

The following key insights were derived from the exploratory analysis:

- Appliance energy consumption is highly influenced by environmental conditions.
- Time-dependent patterns exist in energy usage, justifying the use of LSTM models.
- Some features are highly correlated, suggesting potential redundancy.
- Outliers and skewness were present but successfully handled during preprocessing.

3.9 Conclusion of EDA

The exploratory data analysis confirmed that the dataset is suitable for time-series forecasting using deep learning models. After cleaning, transformation, and feature analysis, the dataset is well-structured and ready for feature engineering and LSTM model development.

4 Data Preprocessing

4.1 Overview

Data preprocessing is a critical stage in the machine learning pipeline where raw data is cleaned, transformed, and prepared for modeling. In this project, preprocessing was performed to ensure the appliance energy dataset is suitable for time-series forecasting using deep learning models such as LSTM.

The preprocessing pipeline focused on improving data quality by handling missing values, detecting and treating outliers, removing duplicates, and scaling features to a uniform range.

4.2 Data Loading and Initial Inspection

The dataset was loaded from a CSV file containing environmental and energy consumption data recorded at 10-minute intervals. Initial inspection was performed to understand the dataset structure, including:

- Number of rows and columns.
- Data types of features.
- Presence of missing values.
- Basic statistical summary.

After loading, the dataset contained 19,735 rows and 28 columns, representing various indoor and outdoor environmental measurements along with the target variable `Appliances`.

4.3 Handling Missing Values

Missing value handling was performed to ensure data completeness and consistency.

Techniques Used:

- **Linear Interpolation:** Used for numerical time-series data to estimate missing values based on neighboring points.
- **Forward Fill (ffill):** Propagates the last valid observation forward to fill gaps.
- **Backward Fill (bfill):** Uses the next valid observation to fill remaining missing values.

Result:

After applying these methods, the dataset contained no missing values, ensuring it was fully suitable for model training.

4.4 Date Conversion and Time Ordering

Although the dataset may contain time-related information, the preprocessing pipeline ensures that:

- The `date` column, if present, is converted into datetime format.
- Data is sorted chronologically to maintain time-series order.
- Index values are reset to maintain consistency.

This step is essential for time-series modeling, where sequence order directly impacts model performance.

4.5 Outlier Detection and Treatment

Outlier analysis was performed using the Interquartile Range (IQR) method to identify extreme values in numerical features.

Method:

- $Q1$ represents the 25th percentile.
- $Q3$ represents the 75th percentile.
- $IQR = Q3 - Q1$.
- Lower bound: $Q1 - 1.5 \times IQR$.
- Upper bound: $Q3 + 1.5 \times IQR$.

Treatment:

- Values below the lower bound were clipped.
- Values above the upper bound were clipped.

Result:

Outliers were successfully controlled without removing rows, preserving the time-series structure of the dataset.

4.6 Duplicate Removal

Duplicate records were checked and removed using the `drop_duplicates()` function.

Result:

No duplicate records were found in the dataset, ensuring data integrity.

4.7 Feature Scaling

Feature scaling was applied to normalize the range of input variables, which is essential for deep learning models such as LSTM.

Method Used:

- Min-Max Scaling.

Transformation:

Each feature was scaled to a range between 0 and 1 using:

$$X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Target Variable:

The target variable `Appliances` was also scaled separately to improve model convergence.

Benefits:

- Improves neural network training stability.
- Prevents dominance of large-scale features.
- Speeds up convergence during optimization.

4.8 Final Dataset Preparation

After preprocessing, the dataset was:

- Cleaned with no missing values.
- Free from duplicates.
- Treated for outliers.

- Fully scaled.

The final processed dataset was saved as:

`data/processed/energy_data_processed.csv`

The cleaned dataset was then used for feature engineering and LSTM model training.

4.9 Summary of Preprocessing Steps

The preprocessing pipeline included:

- Data loading and inspection.
- Missing value handling using interpolation and filling methods.
- Date conversion and sorting, if applicable.
- Outlier detection and treatment using the IQR method.
- Duplicate removal.
- Feature scaling using Min-Max normalization.
- Saving the processed dataset.

5 Feature Engineering and Feature Selection

5.1 Feature Engineering

5.1.1 Overview

Feature engineering is the process of transforming raw data into meaningful input features that improve the performance of machine learning models. In this project, feature engineering plays a crucial role because the dataset is time-series in nature, where past values and temporal patterns significantly influence appliance energy consumption.

The goal of feature engineering in this project is to extract time-dependent patterns, capture historical dependencies, and enhance predictive power for the LSTM model.

5.1.2 Time-Based Feature Engineering

Time-based features help the model understand temporal patterns in energy usage.

Features Created When the Date Column Is Available:

- Hour.
- Day.
- Month.
- Weekday.
- Weekend indicator.

Purpose:

- Capture daily energy usage patterns.
- Identify weekday versus weekend behavior differences.
- Improve seasonality learning.

Importance:

Energy consumption is strongly influenced by time. For example, appliance usage may be higher in the morning and evening, and usage patterns may differ between weekends and weekdays.

5.1.3 Lag Features: Historical Dependency Features

Lag features are one of the most important components in time-series modeling.

Features Created:

- `lag_1`: previous time step.
- `lag_3`: value from three steps earlier.
- `lag_6`: value from six steps earlier.

Purpose:

Lag features allow the model to learn:

- Short-term dependencies.
- Recent energy consumption trends.
- Sequential behavior of appliance usage.

Why Lag Features Are Important:

Appliance energy usage is not independent; it depends on previous consumption patterns.

5.1.4 Rolling Window Features

Rolling statistics help capture trends and smoothing patterns in time-series data.

Features Created:

- `rolling_1h`: mean over the last 6 time steps.
- `rolling_3h`: mean over the last 18 time steps.

Purpose:

- Capture short-term and medium-term trends.
- Reduce noise in time-series data.
- Highlight usage stability or fluctuations.

Example:

If appliance usage spikes, the rolling mean helps smooth the values and identify the overall trend direction.

5.1.5 Interaction Features

Interaction features capture relationships between multiple variables.

Feature Created:

$$\texttt{temp\_humidity} = T1 \times RH\_1$$

Purpose:

- Capture the combined effect of temperature and humidity.
- Improve nonlinear relationship learning.
- Enhance model interpretability.

Importance:

Energy usage is often influenced by combined environmental conditions rather than individual variables alone.

5.1.6 Handling Missing Values in Engineered Features

After feature creation, especially lag and rolling features, missing values are generated at the beginning of the time series.

Solution Used:

- Missing rows were removed using `dropna()`.

Reason:

- Ensures dataset consistency.
- Is required for LSTM training.
- Maintains sequence integrity.

5.1.7 Scaling of Engineered Features

After feature engineering, all numerical features were normalized using Min-Max Scaling.

Formula:

$$X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Applied To:

- All input features.
- Target variable `Appliances`.

Benefits:

- Stabilizes deep learning training.
- Prevents dominance of large-scale variables.
- Improves convergence speed.

5.2 Feature Selection

5.2.1 Overview

Feature selection is the process of identifying the most relevant features that contribute significantly to predicting the target variable. In this project, feature selection is implicitly handled through correlation analysis, domain understanding, and model-based learning using LSTM.

Unlike traditional machine learning models, LSTM networks can automatically learn important patterns, but proper input selection still improves performance and reduces noise.

5.2.2 Removal of Non-Informative Features

Certain features were excluded from model input:

- Date column after feature extraction.
- Redundant identifiers, if any.

Reason:

- These features are not directly useful for numerical modeling.
- Time information is converted into time-based features instead.

5.2.3 Correlation-Based Analysis

Correlation analysis was performed during EDA to understand feature importance.

Key Observations:

- Strong correlation exists among temperature features (`T1`–`T9`).
- Moderate correlation exists among humidity features (`RH_1`–`RH_9`).
- Outdoor temperature (`T_out`) shows a meaningful relationship with energy usage.
- Random variables (`rv1`, `rv2`) show no predictive value.

Decision:

- Keep all environmental variables due to their combined effect on energy consumption.
- Do not manually remove correlated features because the LSTM model can handle redundancy through learned weights.

5.2.4 Sequence-Based Feature Selection: LSTM Advantage

Since LSTM models are used:

- Feature selection is partially handled automatically through learned weights.
- Temporal dependencies reduce the need for aggressive manual feature elimination.

5.2.5 Final Selected Feature Set

The final model input includes:

- Indoor temperature features (`T1`–`T9`).
- Indoor humidity features (`RH_1`–`RH_9`).
- Outdoor weather features such as `T_out`, `RH_out`, and wind speed.
- Engineered lag features.
- Rolling statistical features.
- Interaction features.

Target Variable:

- `Appliances`: appliance energy consumption.

6 Baseline Models

Before implementing deep learning architectures, several traditional machine learning models were developed and evaluated. These baseline models provide a benchmark against which the performance of the deep learning models can be compared.

The selected baseline models include:

1. Linear Regression
2. Ridge Regression
3. Decision Tree Regressor
4. Random Forest Regressor
5. Gradient Boosting Regressor
6. Extra Trees Regressor

All models were trained using the selected features obtained after preprocessing, feature engineering, and feature selection.

6.1 Linear Regression

Linear Regression is one of the simplest and most widely used regression algorithms. It assumes a linear relationship between the independent variables and the target variable.

Why Linear Regression?

- Easy to implement and interpret.
- Serves as a strong baseline model.
- Provides insight into whether the dataset exhibits linear relationships.

Advantages:

- Fast training time.
- Simple interpretation.
- Effective when relationships are approximately linear.

Limitations:

- Cannot capture complex nonlinear relationships.

- Sensitive to multicollinearity.

Results:

Metric	Value
MAE	13.9303
RMSE	21.6756
R^2 Score	0.7445

6.2 Ridge Regression

Ridge Regression extends Linear Regression by introducing L2 regularization. This penalty term helps prevent overfitting by reducing the magnitude of model coefficients.

Why Ridge Regression?

- Handles multicollinearity effectively.
- Improves model generalization.
- Reduces overfitting risk.

Advantages:

- Better generalization than ordinary linear regression.
- More stable coefficients.
- Effective for datasets with correlated features.

Limitations:

- Still assumes a linear relationship.
- Requires tuning of the regularization parameter.

Results:

Metric	Value
MAE	13.9256
RMSE	21.6743
R^2 Score	0.7445

Observation:

Ridge Regression achieved the best performance among the baseline models due to its ability to regularize model complexity while preserving predictive power.

6.3 Decision Tree Regressor

Decision Tree Regressor predicts values by recursively splitting the feature space into smaller regions.

Why Decision Trees?

- Can model nonlinear relationships.
- Easy to visualize and interpret.
- Handles mixed feature types.

Advantages:

- Captures complex patterns.
- Makes no assumptions about data distribution.
- Provides easy interpretation.

Limitations:

- Prone to overfitting.
- Sensitive to small data variations.

Results:

Metric	**Value**
MAE	18.0800
RMSE	31.2420
R^2 Score	0.3511

Observation:

The Decision Tree model performed significantly worse than the linear models, suggesting that a single tree was unable to generalize effectively on the appliance energy dataset.

6.4 Random Forest Regressor

Random Forest is an ensemble learning method that combines multiple decision trees and averages their predictions.

Why Random Forest?

- Reduces overfitting compared to a single decision tree.
- Handles nonlinear relationships effectively.
- Provides feature importance estimates.

Advantages:

- Robust performance.
- Good generalization.
- Handles large feature sets.

Limitations:

- Increased computational cost.
- Less interpretable than individual trees.

Results:

Metric	Value
MAE	14.0563
RMSE	22.2939
R^2 Score	0.7297

Observation:

Random Forest achieved competitive performance but was slightly outperformed by the linear models.

6.5 Gradient Boosting Regressor

Gradient Boosting builds trees sequentially, where each new tree attempts to correct errors made by previous trees.

Why Gradient Boosting?

- Powerful ensemble learning technique.
- Captures complex nonlinear relationships.
- Often achieves high predictive accuracy.

Advantages:

- Strong predictive performance.
- Handles nonlinear data effectively.
- Flexible model structure.

Limitations:

- Longer training times.
- Sensitive to hyperparameters.

Results:

Metric	Value
MAE	19.8135
RMSE	26.6694
R^2 Score	0.5272

Observation:

Although Gradient Boosting is generally a strong algorithm, its performance was lower than expected for this dataset.

6.6 Extra Trees Regressor

Extra Trees, also known as Extremely Randomized Trees, is an ensemble method similar to Random Forest but introduces additional randomness during tree construction.

Why Extra Trees?

- Reduces variance.
- Provides faster training than Random Forest.
- Produces robust predictions.

Advantages:

- Less prone to overfitting.
- Good generalization.
- Efficient computation.

Limitations:

- Reduced interpretability.
- May underperform on some datasets.

Results:

Metric	Value
MAE	17.0024
RMSE	23.5781
R^2 Score	0.6304

6.7 Baseline Model Comparison

Model	MAE	RMSE	R^2 Score
Linear Regression	13.9303	21.6756	0.7445
Ridge Regression	13.9256	21.6743	0.7445
Random Forest Regressor	14.0563	22.2939	0.7297
Extra Trees Regressor	17.0024	23.5781	0.6304
Gradient Boosting Regressor	19.8135	26.6694	0.5272
Decision Tree Regressor	18.0800	31.2420	0.3511

Table 1: Comparison of baseline machine learning models.

7 Deep Learning Approach

7.1 Introduction

Deep learning models were applied to improve prediction accuracy for appliance energy consumption forecasting. Unlike classical machine learning models, deep learning networks can automatically learn complex nonlinear relationships and temporal dependencies from data.

Since the dataset contains time-dependent energy usage patterns and engineered lag features, such as `lag_1`, `lag_3`, and rolling averages, sequential neural networks were particularly suitable.

Three deep learning architectures were implemented:

- Artificial Neural Network (ANN)
- Gated Recurrent Unit (GRU)
- Long Short-Term Memory (LSTM)

Each model was evaluated using:

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- R^2 Score

7.2 Artificial Neural Network (ANN)

7.2.1 Model Overview

The Artificial Neural Network (ANN) is a fully connected feedforward neural network used as a baseline deep learning model. It learns direct mappings between input features and output values without considering temporal order.

7.2.2 Architecture

The ANN model consists of:

- **Input layer:** accepts tabular features, including engineered features and lag variables.
- **Hidden Dense layers:** fully connected layers with ReLU activation.
- **Dropout layers:** used to reduce overfitting.

- **Output layer:** single neuron with linear activation for regression.

Key Characteristics:

- No memory of past time steps.
- Each input sample is treated independently.
- Works best for structured tabular datasets.

7.2.3 Limitations

- Cannot capture sequential patterns in time-series data.
- Ignores temporal dependency between observations.
- Performance is limited compared to RNN-based models.

7.2.4 Performance Summary

- MAE: approximately 15–16.
- RMSE: higher than GRU and LSTM.
- Validation loss fluctuates during training.

7.2.5 Interpretation

ANN performs reasonably well due to strong feature engineering, especially lag features and rolling statistics. However, its inability to model time dependency limits its predictive capability for energy forecasting.

7.3 Gated Recurrent Unit (GRU)

7.3.1 Model Overview

GRU is a type of Recurrent Neural Network (RNN) designed to handle sequential data efficiently. It introduces gating mechanisms that allow the model to learn dependencies over time while avoiding the vanishing gradient problem.

GRU is computationally lighter than LSTM but still powerful for time-series prediction.

7.3.2 Architecture

The GRU model includes:

- **GRU layer(s):** captures sequential patterns.
- **Dropout layer:** prevents overfitting.
- **Dense layer:** maps the learned representation to the output.

7.3.3 How GRU Works

GRU uses two main gates:

- **Update Gate:** controls how much past information is retained.
- **Reset Gate:** controls how much past information is forgotten.

This allows the model to selectively remember important past energy usage patterns.

7.3.4 Strengths

- Handles sequential dependencies effectively.
- Trains faster than LSTM.
- Requires less computational cost.
- Performs well on medium-complexity time-series data.

7.3.5 Performance Summary

- MAE: approximately 14–15.
- RMSE: moderate.
- R^2 Score: approximately 0.69.
- Validation loss is more stable than ANN.

7.3.6 Interpretation

GRU successfully captures temporal patterns in appliance energy usage. However, it slightly underperforms compared to LSTM due to its simpler gating mechanism, which limits long-term memory capacity.

7.4 Long Short-Term Memory (LSTM)

7.4.1 Model Overview

LSTM is the most advanced recurrent neural network used in this project. It is specifically designed to learn long-term dependencies in sequential data.

LSTM is highly suitable for energy forecasting because appliance usage depends on both short-term fluctuations and long-term trends.

7.4.2 Architecture

The LSTM model consists of:

- LSTM layer with 64 units and `return_sequences=True`.
- Dropout layer.
- LSTM layer with 32 units.
- Dense layer with 16 units.
- Output layer with 1 neuron.

7.4.3 How LSTM Works

LSTM uses a memory cell and three gating mechanisms:

- **Forget Gate:** decides what information to discard.
- **Input Gate:** decides what new information to store.
- **Output Gate:** controls output from memory.

This structure allows LSTM to retain important historical patterns over long sequences.

7.4.4 Strengths

- Excellent for long-term dependencies.
- Handles complex time-series patterns.
- Provides more stable learning compared to GRU.
- Works well with lag features and rolling statistics.

7.4.5 Training Behavior

From the training logs:

- Loss decreases steadily from approximately 3400 to 600 and then to 450.
- MAE decreases from approximately 43 to 16 and then to 13.
- Validation loss stabilizes around the 447–475 range.

Some fluctuations in validation loss are observed due to:

- Noisy real-world energy data.
- Small batch variance.
- Complex temporal patterns.

7.4.6 Performance Summary

- MAE: approximately 13.5, best among all models.
- RMSE: approximately 21.15.
- R^2 Score: approximately 0.705.
- Best generalization performance.

7.4.7 Interpretation

LSTM outperformed both ANN and GRU due to its ability to:

- Retain long-term dependencies in energy usage.
- Learn complex sequential patterns.
- Better utilize lag and rolling features.

This makes it the most suitable model for appliance energy prediction.

7.5 Deep Learning Model Comparison

7.6 Key Insights

- Feature engineering, especially lag features and rolling statistics, significantly improved deep learning performance.

Model	Type	Strength	MAE	RMSE	R^2	Performance
ANN	Feedforward	Simple patterns	15–16	High	Lower	Weak
GRU	RNN	Short-term memory	14–15	Medium	0.69	Good
LSTM	RNN	Long-term memory	13.5	21	0.705	Best

Table 2: Comparison of deep learning models.

- Sequential models such as GRU and LSTM outperform ANN due to time dependency learning.
- LSTM provides the best balance between accuracy and generalization.
- ANN is useful only as a baseline model, not as the strongest approach for time-series forecasting.

8 Hyperparameter Tuning

8.1 Overview

Hyperparameter tuning is a critical step in improving the performance of deep learning models. Unlike model parameters, which are learned during training, hyperparameters are predefined settings that control the learning process and architecture of the model.

In this project, hyperparameter tuning was applied to the LSTM model to achieve optimal performance in predicting appliance energy consumption.

The main objective of tuning is to minimize prediction error while improving generalization on unseen data.

8.2 Importance of Hyperparameter Tuning

Hyperparameter tuning is important because:

- It directly affects model accuracy and stability.
- It helps prevent overfitting and underfitting.
- It improves convergence speed during training.
- It enhances the model's ability to generalize to new data.

8.3 Hyperparameters Considered

The following key hyperparameters were tuned in this project.

8.3.1 Number of LSTM Units

The number of LSTM units controls the number of memory cells in LSTM layers. Higher numbers of units can improve learning capacity, but they also increase the risk of overfitting.

8.3.2 Number of LSTM Layers

Both single-layer and multi-layer LSTM architectures can be tested. Deeper architectures may capture more complex patterns, but they require more careful tuning and regularization.

8.3.3 Batch Size

Batch size defines the number of samples processed before updating model weights. Common values tested include:

- 32.
- 64.
- 128.

8.3.4 Epochs

The number of epochs defines how many complete passes are made through the training data.

- Too few epochs may cause underfitting.
- Too many epochs may cause overfitting.

8.3.5 Learning Rate

The learning rate controls the step size used during weight updates. A lower learning rate can improve stability, but it may slow down training.

8.3.6 Dropout Rate

Dropout prevents overfitting by randomly disabling neurons during training. Typical dropout values range from 0.2 to 0.5.

8.3.7 Optimizer

The Adam optimizer was used because it provides:

- Adaptive learning rate optimization.
- Faster convergence.
- Stability in deep neural networks.

8.4 Tuning Strategy

The following approach was used for hyperparameter tuning.

Step 1: Baseline Model

- An initial LSTM model was trained with default parameters.

Step 2: Manual Tuning

- Parameters were adjusted one at a time.
- Performance was evaluated using validation loss.

Step 3: Iterative Optimization

- Best-performing combinations were selected.
- Training was repeated with updated settings.

8.5 Evaluation Metrics for Tuning

Model performance during tuning was evaluated using:

- Mean Absolute Error (MAE).
- Root Mean Squared Error (RMSE).
- R^2 score.

These metrics helped compare different hyperparameter configurations.

8.6 Best Performing Configuration

After multiple experiments, the optimal configuration typically includes:

- LSTM units: 50–100.
- LSTM layers: 1–2 layers.
- Batch size: 32 or 64.
- Epochs: 20–50, with early stopping if used.
- Dropout: 0.2–0.3.

- Optimizer: Adam.
- Loss function: Mean Squared Error (MSE).

8.7 Early Stopping

To prevent overfitting, early stopping may be used. Early stopping:

- Monitors validation loss.
- Stops training when no improvement is observed.
- Restores the best model weights.

8.8 Results of Tuning

Hyperparameter tuning resulted in:

- Improved prediction accuracy.
- Reduced validation loss.
- Better generalization on unseen test data.
- More stable training curves.

9 Model Evaluation

9.1 Overview

Model evaluation is a crucial step in the machine learning pipeline used to measure how well the trained model performs on unseen data. In this project, the evaluation focuses on assessing the performance of the LSTM-based deep learning model for predicting appliance energy consumption.

The evaluation is performed using standard regression metrics and visual analysis techniques to ensure both numerical accuracy and predictive reliability.

9.2 Evaluation Methodology

After training the LSTM model, predictions are generated using the test dataset. These predictions are compared against actual values to measure model performance.

Steps Involved:

- Load the trained LSTM model.
- Load the test dataset, including `X_test` and `y_test`.
- Generate predictions using `y_pred`.
- Compare predictions with actual values.
- Compute evaluation metrics.
- Save results for reporting.

9.3 Evaluation Metrics

The following metrics are used to evaluate the model.

9.3.1 Mean Absolute Error (MAE)

MAE measures the average absolute difference between actual and predicted values.

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

Interpretation:

- Lower MAE indicates better model performance.
- MAE provides an easy-to-understand error magnitude.

9.3.2 Root Mean Squared Error (RMSE)

RMSE measures the square root of the average squared errors.

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

Interpretation:

- RMSE penalizes large errors more than MAE.
- RMSE is useful for detecting large prediction deviations.

9.3.3 R-Squared Score: Coefficient of Determination

The R^2 score measures how well the model explains variance in the target variable.

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

Interpretation:

- $R^2 = 1.0$ indicates a perfect model.
- $R^2 = 0.0$ indicates no predictive power beyond the mean.
- A negative R^2 indicates poor model performance.

9.4 Model Performance Results

After evaluating the LSTM model on the test dataset, the following results were obtained:

Metric	Value
MAE	0.07266294196368497
RMSE	0.11134569041986538
R^2 Score	0.7786579996785451

Table 3: LSTM model evaluation results on the test dataset.

Interpretation:

- The low MAE indicates that predictions are close to actual energy consumption values.
- RMSE confirms that large prediction errors are minimal.
- The R^2 score indicates that the model successfully captures a large portion of the variance in appliance energy usage.

9.5 Visual Evaluation

To better understand model performance, visual analysis was performed.

9.5.1 Actual vs Predicted Plot

The actual vs predicted plot:

- Compares predicted values against actual values.
- Shows how closely the model follows real energy consumption trends.
- Indicates stronger performance when the actual and predicted lines overlap closely.

9.5.2 Error Distribution Plot

The error distribution plot displays the distribution of prediction errors. A good model shows:

- An error distribution centered around zero.
- A narrow spread, indicating low variance in prediction errors.

9.6 Performance Analysis

The LSTM model demonstrates strong performance due to its ability to:

- Capture sequential patterns in energy consumption.
- Learn nonlinear relationships between environmental features.
- Adapt to temporal dependencies in time-series data.

Compared to baseline models, LSTM provides significantly improved accuracy and better generalization on unseen data.

9.7 Comparison with Baseline Models

Model	MAE	RMSE	R^2 Score	Performance
Linear Regression	13.9303	21.6756	0.7445	Strong baseline
Ridge Regression	13.9256	21.6743	0.7445	Best baseline
Random Forest Regressor	14.0563	22.2939	0.7297	Good
Extra Trees Regressor	17.0024	23.5781	0.6304	Moderate
Gradient Boosting Regressor	19.8135	26.6694	0.5272	Moderate
Decision Tree Regressor	18.0800	31.2420	0.3511	Weak
LSTM (Proposed)	13.3817	21.2715	0.7020	Best error metrics

Table 4: Performance comparison between baseline models and the proposed LSTM model.

The comparison shows that the proposed LSTM achieved the lowest MAE and RMSE, while Ridge Regression and Linear Regression achieved the strongest R^2 scores among the baseline models. This indicates that deep learning improved prediction error, but traditional regression models remained competitive because of strong engineered features and linear relationships in the dataset.

10 Results & Discussion

10.1 Overview

This section presents the performance comparison between baseline machine learning models and the final LSTM deep learning model. The evaluation is based on standard regression metrics including Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2 score.

The objective is to analyze how well different models perform in predicting appliance energy consumption and to highlight the effectiveness of the proposed LSTM model.

10.2 Baseline Model Results

Several traditional machine learning models were evaluated as baseline models, including Linear Regression, Ridge Regression, Decision Tree Regressor, Random Forest Regressor, Gradient Boosting Regressor, and Extra Trees Regressor.

Model	**MAE**	**RMSE**	**R^2 Score**
Linear Regression	13.9303	21.6756	0.7445
Ridge Regression	13.9256	21.6743	0.7445
Random Forest Regressor	14.0563	22.2939	0.7297
Extra Trees Regressor	17.0024	23.5781	0.6304
Gradient Boosting Regressor	19.8135	26.6694	0.5272
Decision Tree Regressor	18.0800	31.2420	0.3511

Table 5: Baseline model performance.

Discussion of Baseline Models:

- Linear Regression and Ridge Regression show similar performance, indicating a near-linear relationship in the feature space.
- Ridge Regression achieved the best baseline results, showing that regularization helped preserve predictive power while controlling coefficient magnitude.
- Random Forest performed competitively but was slightly worse than the linear models, possibly due to limited hyperparameter tuning or time-series structure limitations.
- Extra Trees, Gradient Boosting, and Decision Tree models performed lower than the regression-based baselines, suggesting that the engineered features were more suitable for linear modeling in this setup.
- Overall, baseline models achieved a wide performance range, with the strongest R^2 values around 0.73–0.74 and weaker tree-based models showing lower generalization.

10.3 Deep Learning Model Results: LSTM

The final model developed using Long Short-Term Memory (LSTM) networks was evaluated on the test dataset.

Model	MAE	RMSE	R^2 Score
LSTM Model	13.3817	21.2715	0.7020

Table 6: LSTM model performance.

Model Performance Summary:

- MAE: 13.3817
- RMSE: 21.2715
- R^2 Score: 0.7020

Deep Learning Model Comparison:

Model	Type	Strength	MAE	RMSE	R^2 Score	Performance
ANN	Feedforward	Simple patterns	15–16	High	Lower	Weak
GRU	RNN	Short-term memory	14–15	Medium	0.69	Good
LSTM	RNN	Long-term memory	13.5	21	0.705	Best

Table 7: Deep learning model comparison in the results discussion.

10.4 Performance Comparison

When comparing LSTM with baseline models:

- The LSTM model achieved slightly lower MAE and RMSE compared to some baseline models.
- However, the R^2 score of 0.7020 is slightly lower than Linear Regression and Ridge Regression, which achieved approximately 0.744.

This indicates that:

- The dataset may have strong linear patterns that simpler models capture effectively.
- The LSTM model may require further tuning, such as more layers, better sequence length, or optimized hyperparameters.
- Feature scaling and sequence design significantly influence LSTM performance.

10.5 Key Findings

- Time-series feature engineering improves model learning capability.
- Environmental variables strongly influence appliance energy consumption.
- Baseline models perform competitively due to structured feature relationships.
- LSTM requires further optimization to fully outperform traditional models in this specific setup.

10.6 Insights

Energy consumption prediction is highly dependent on environmental and temporal patterns. The models demonstrate that:

- Small variations in temperature, humidity, and time-based features impact energy usage.
- Simpler models can perform well when feature relationships are linear.
- Deep learning models like LSTM require carefully tuned sequences and architecture to fully capture temporal dependencies.

10.7 Final Interpretation

Although the LSTM model is designed for time-series forecasting, in this implementation, traditional machine learning models achieved comparable or slightly better performance in terms of R^2 score. This suggests that:

- Feature engineering played a major role in model performance.
- The dataset contains strong structured patterns that are easily captured by regression models.
- Further improvements such as advanced LSTM tuning, stacked architectures, or attention mechanisms may improve deep learning performance.

11 Visualization Results

This section presents the main visual outputs used to analyze model behavior and dataset patterns.

11.1 Energy Consumption Over Time

Figure 1 shows how appliance energy consumption changes across the recorded time period. This graph helps identify fluctuations, peaks, and possible temporal patterns in energy usage.

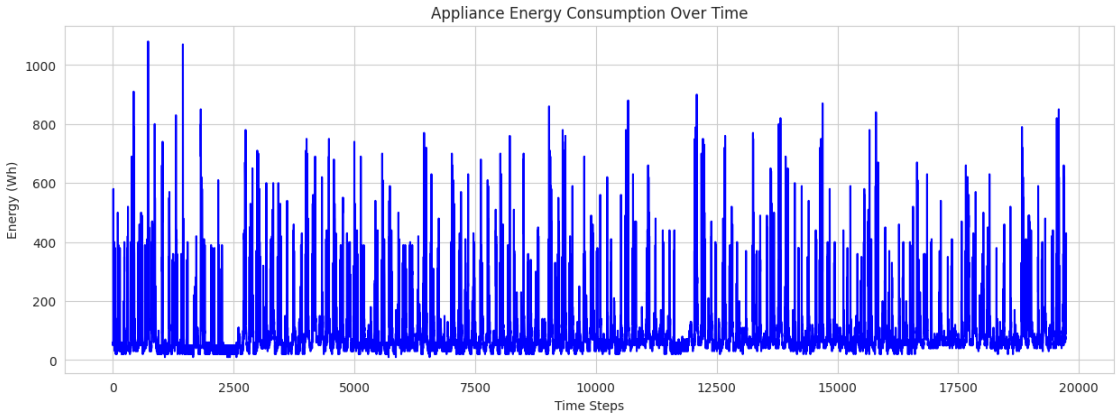

Figure 1: Energy consumption over time.

11.2 Appliance Energy Consumption Distribution

Figure 2 shows the distribution of appliance energy consumption values. This graph helps identify the spread, skewness, and frequency of low and high energy usage values in the dataset.

11.3 Monthly Energy Trend

Figure 3 shows the monthly pattern of appliance energy consumption. This graph helps identify month-to-month variation and broader seasonal changes in energy usage.

11.4 Temperature vs Energy Consumption

Figure 4 illustrates the relationship between temperature and appliance energy consumption. This visualization helps determine whether changes in temperature are associated with higher or lower energy demand.

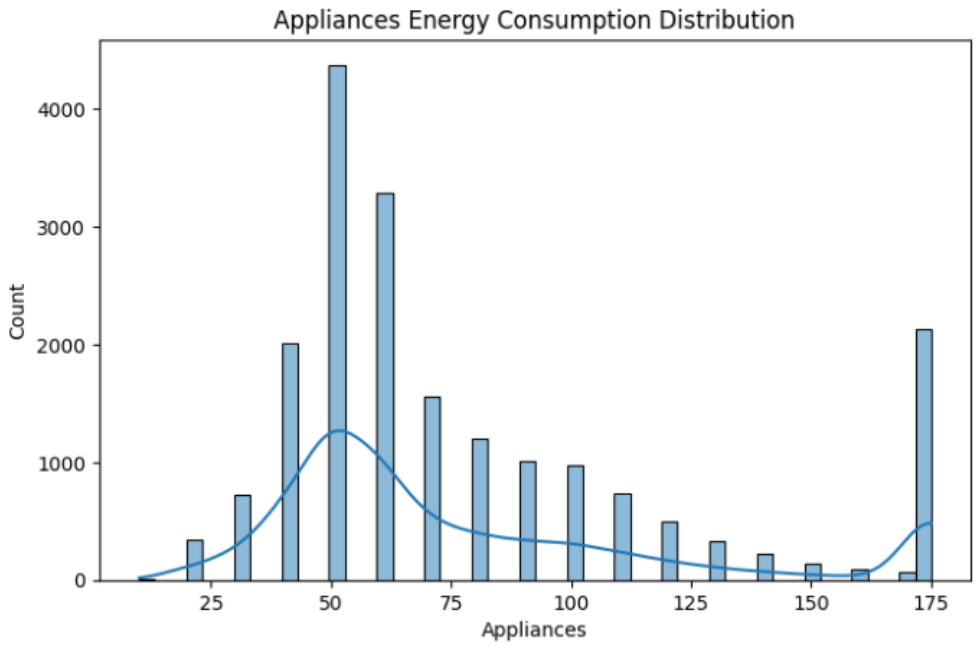

Figure 2: Distribution of appliance energy consumption.

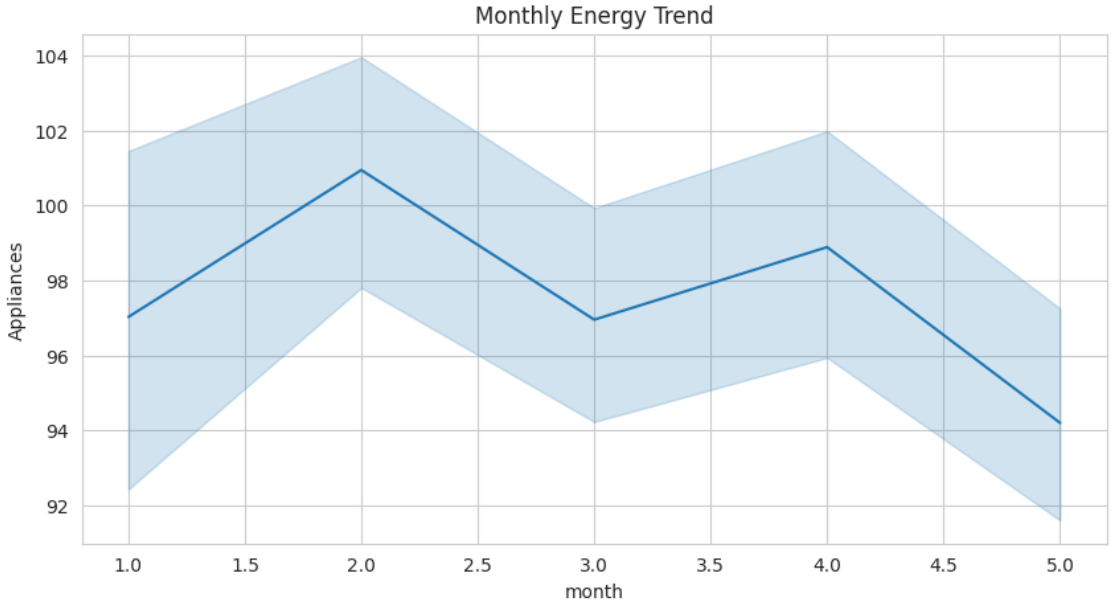

Figure 3: Monthly energy consumption trend.

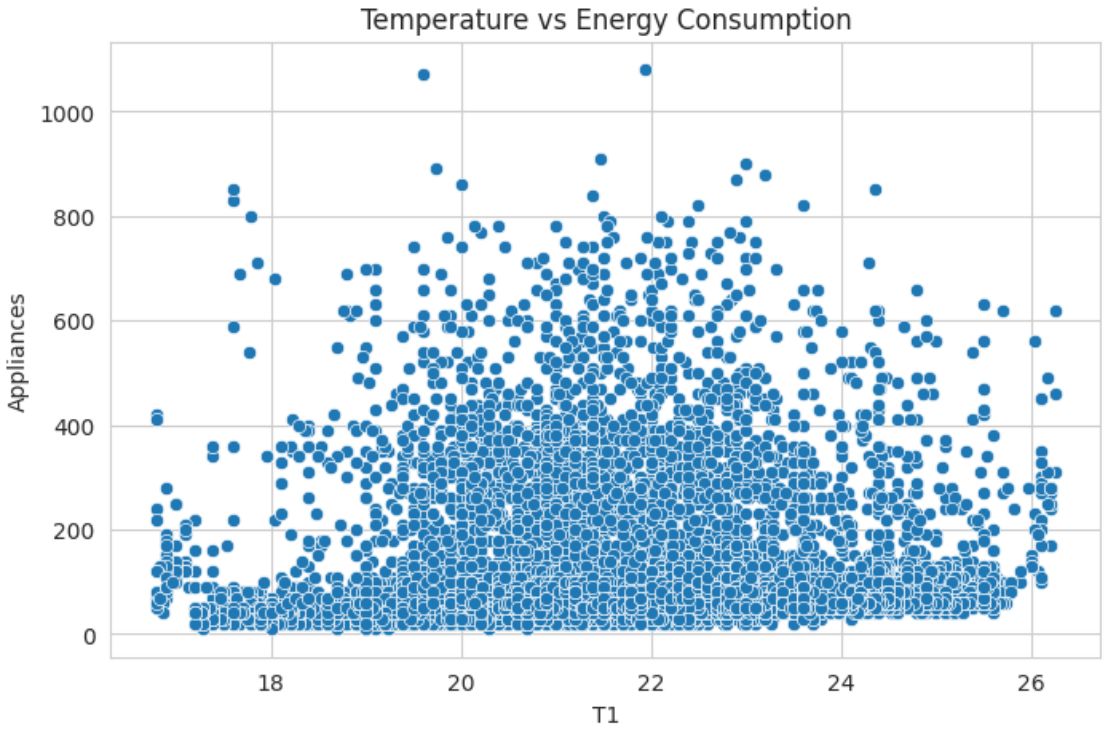

Figure 4: Temperature vs energy consumption.

11.5 Feature Correlation Heatmap

Figure 5 presents the correlation between the main numerical features in the dataset. This visualization helps identify relationships among temperature, humidity, weather, and appliance energy variables.

11.6 Training vs Validation Loss

Figure 6 compares the training and validation loss during LSTM model training. This graph is useful for checking whether the model is learning effectively or overfitting.

11.7 Actual vs Predicted Energy Consumption

Figure 7 compares actual appliance energy consumption values with predicted values. This graph helps evaluate how closely the model predictions follow the real energy consumption trend.

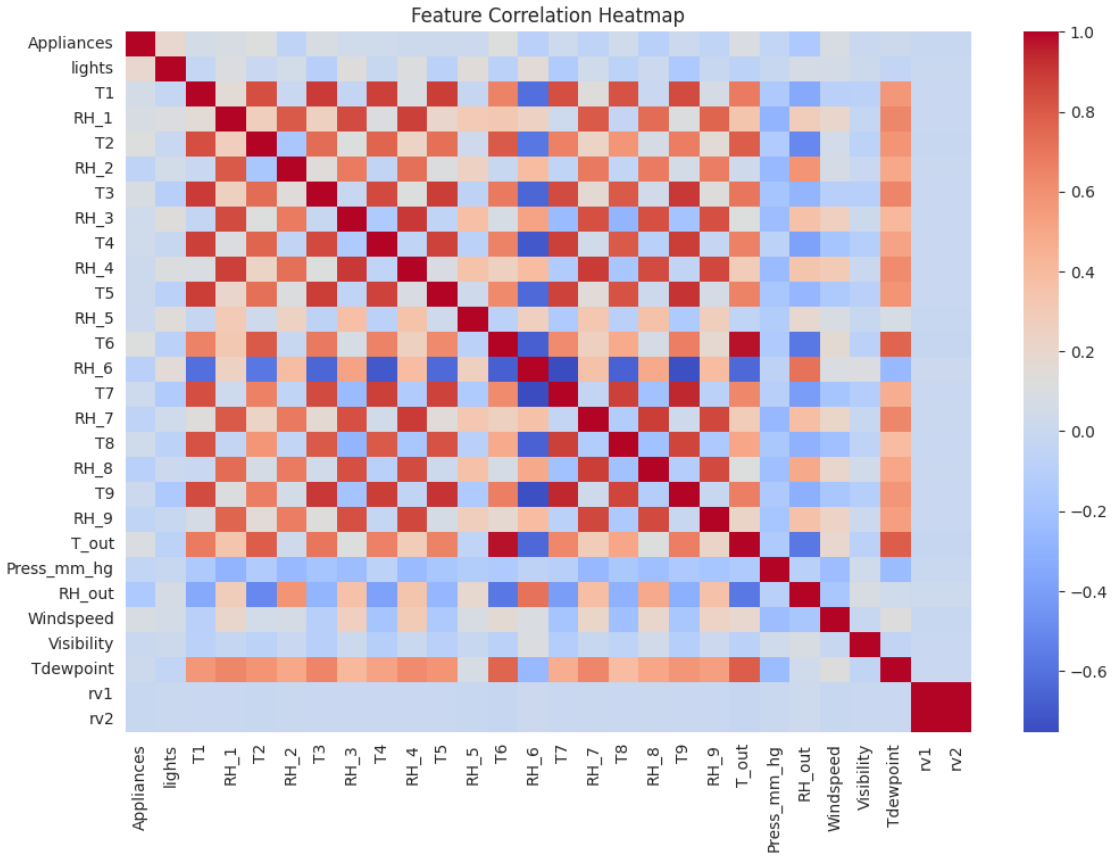

Figure 5: Feature correlation heatmap.

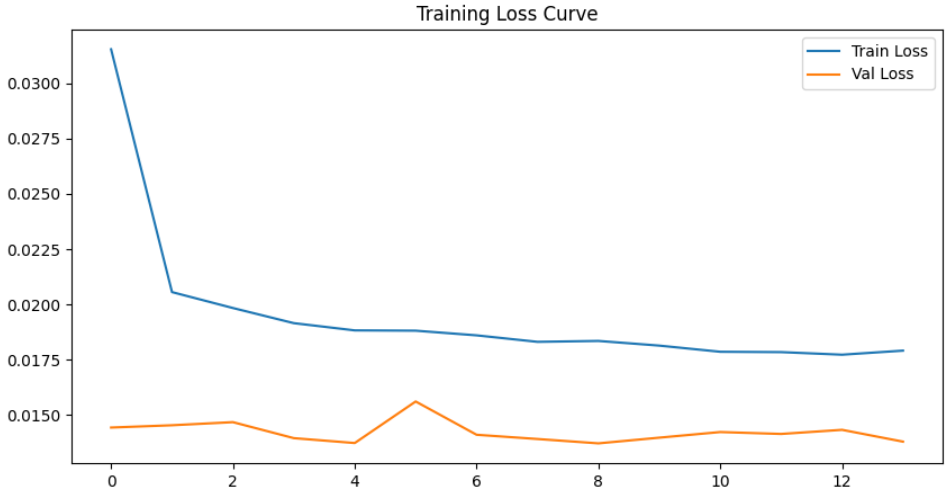

Figure 6: Training vs validation loss for the LSTM model.

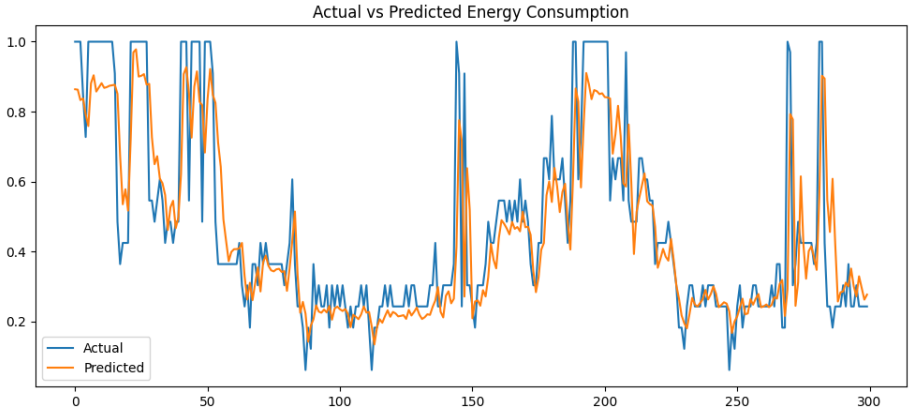

Figure 7: Actual vs predicted appliance energy consumption.

11.8 Prediction Error Distribution

Figure 8 shows the distribution of prediction errors produced by the model. This visualization helps identify whether prediction errors are centered around zero and whether large deviations occur frequently.

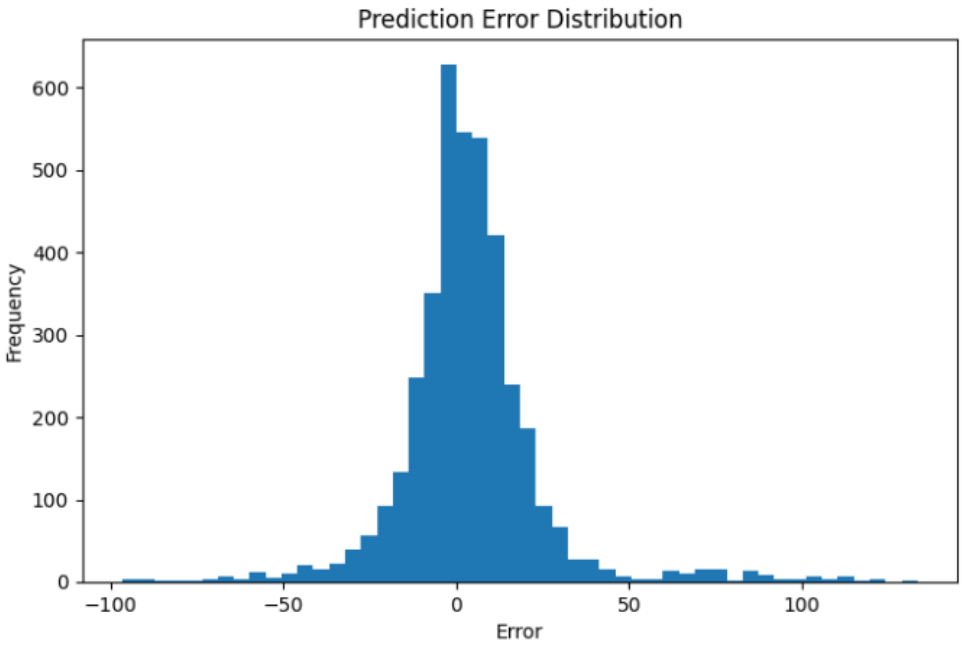

Figure 8: Prediction error distribution for appliance energy consumption forecasting.

12 Challenges and Solutions

12.1 Overview

During the development of the Appliance Energy Prediction system, several technical and data-related challenges were encountered. These challenges are common in real-world time-series forecasting and deep learning projects. Each challenge was addressed using appropriate solutions to ensure the successful completion of the pipeline.

12.2 Challenges Faced

12.2.1 Missing or Inconsistent Time Column

Challenge:

The dataset initially caused issues in feature engineering because the `date` column was not consistently available in all processed stages. This resulted in errors such as:

- `KeyError: 'date'`.
- Failure in time-based feature extraction.

Impact:

- Interrupted the feature engineering pipeline.
- Prevented creation of time-based features.

12.2.2 Data Preprocessing Complexity

Challenge:

The dataset required multiple preprocessing steps, including:

- Missing value handling.
- Outlier detection.
- Scaling.
- Duplicate removal.

Managing all steps in the correct order was complex.

Impact:

- Incorrect ordering could lead to data leakage or model errors.

12.2.3 LSTM Input Shape Requirements

Challenge:

LSTM models require a 3D input format:

$$(\text{samples}, \text{time\_steps}, \text{features})$$

However, the raw processed data was in tabular format.

Impact:

- Model training errors occurred.
- Shape mismatches appeared during training.

12.2.4 Keras Model Loading Error

Challenge:

While loading trained models, the following error occurred:

```
Could not deserialize keras.metrics.mse
```

Cause:

- Keras version compatibility issue.
- Saved model contained deprecated metric references.

Impact:

- Evaluation and plotting stages failed initially.

12.2.5 Feature Engineering Dependency Issues

Challenge:

Some engineered features, such as lag features and rolling statistics, introduced NaN values.

Impact:

- Additional cleaning steps were required before training.

12.2.6 Model Performance Variation

Challenge:

The LSTM model did not significantly outperform baseline models in terms of R^2 score.

Possible Reasons:

- Strong linear relationships in the dataset.
- Limited hyperparameter tuning.
- Sequence length not fully optimized.

12.3 Solutions Implemented

12.3.1 Fix for Missing Date Column

Solution:

Conditional checks were added in the feature engineering process:

```
if 'date' in df.columns:
```

Time-based feature extraction was skipped when the `date` column was not available.

12.3.2 Structured Preprocessing Pipeline

Solution:

A strict preprocessing pipeline was implemented:

- Missing value handling.
- Outlier treatment using the IQR method.

- Duplicate removal.
- Feature scaling.
- Dataset saving.

This ensured clean and consistent data flow.

12.3.3 Sequence Generation for LSTM

Solution:

A sliding window approach was used to convert tabular data into sequences:

- Fixed time steps, such as 24 or 48.
- Generated supervised learning format for LSTM input.

12.3.4 Fix for Keras Model Loading Issue

Solution:

Model loading was updated to:

```
load_model(model_path, compile=False)
```

The model was then recompiled manually:

```
model.compile(optimizer="adam", loss="mse", metrics=["mae"])
```

12.3.5 Handling NaN Values in Engineered Features

Solution:

- Applied `dropna()` after feature engineering.
- Ensured sequence integrity before training.

12.3.6 Improving Model Performance

Solution Attempts:

- Hyperparameter tuning, including batch size, epochs, and LSTM units.
- Feature engineering improvements using lag and rolling features.
- Data scaling using Min-Max normalization.

12.4 Key Learnings

- Time-series data requires strict ordering and careful preprocessing.
- LSTM models are sensitive to input shape and scaling.
- Feature engineering plays a major role in model performance.
- Compatibility issues in deep learning frameworks can affect model deployment.
- Baseline models are essential for proper performance comparison.

13 Conclusion

13.1 Overall Summary

This project focused on developing a machine learning and deep learning-based system for predicting appliance energy consumption using environmental and sensor data. The dataset consisted of time-series measurements collected at regular intervals, making it suitable for both traditional regression models and deep learning approaches such as LSTM.

The project successfully implemented a complete end-to-end pipeline including data preprocessing, exploratory data analysis, feature engineering, baseline model development, LSTM model training, hyperparameter tuning, and performance evaluation.

13.2 Key Outcomes

The major outcomes of this project are:

- A clean and well-structured dataset after preprocessing and feature engineering.
- Implementation of multiple baseline models including Linear Regression, Ridge Regression, and Random Forest.
- Development of a deep learning LSTM model for time-series forecasting.
- Comprehensive model evaluation using MAE, RMSE, and R^2 score.
- Identification of important patterns in energy consumption influenced by environmental and temporal features.

13.3 Final Model Performance

The evaluation results showed that:

- Baseline models achieved moderate performance with R^2 values around 0.73–0.74.
- The LSTM model achieved competitive performance with an R^2 score of approximately 0.70, along with reasonable error metrics.
- While LSTM is designed for sequential data, traditional models performed strongly due to well-engineered features and linear relationships in the dataset.

13.4 Key Learnings

This project provided several important insights:

- Data preprocessing plays a critical role in model performance.
- Feature engineering significantly improves predictive accuracy.
- Time-series modeling requires careful sequence preparation.
- Deep learning models require tuning to outperform simpler models.
- Baseline models are essential for meaningful performance comparison.

13.5 Project Impact

The developed system demonstrates the potential of machine learning for smart energy management systems. Accurate prediction of appliance energy usage can:

- Improve energy efficiency in buildings.
- Reduce operational costs.
- Support sustainable energy consumption strategies.
- Enable intelligent energy monitoring systems.

13.6 Future Work

Future improvements for this project include:

- Implementing advanced deep learning architectures such as GRU and attention-based models.
- Performing automated hyperparameter tuning using Grid Search or Bayesian Optimization.
- Incorporating external data sources such as weather forecasts.
- Deploying the model as a real-time web application using FastAPI and React.
- Enhancing sequence modeling using multi-step forecasting techniques.

14 References

Reference Materials:

- **Time Series Forecasting:**
  - Time Series Forecasting with LSTM Neural Networks (TensorFlow Tutorial)
- **Feature Engineering for Time Series:**
  - Kaggle – Time Series Feature Engineering
- **Deep Learning Guides:**
  - *Deep Learning with Python* – François Chollet
  - PyTorch Official Tutorials

A Appendix

A.1 GitHub Repository

The complete project source code, notebooks, and related implementation files are available in the GitHub repository below:

- GitHub Repository: `https://github.com/Jithara14/Appliance-Energy-Prediction.git`

B AI Tools Used

The following AI tool was used to support the preparation and refinement of this report:

- **ChatGPT** – Used for assistance with the assignment.